



Hello, everyone, and we...
TXT 17.07KB

make notes out of this transcript for me explain all these topics properly
in english with deep concept and code explanation

Machine Learning Foundations: Supervised Learning — Regression

1. Introduction to Supervised Learning

Supervised learning is the **most common paradigm** in machine learning. When people say "machine learning," they usually mean supervised learning unless they specify otherwise (e.g., unsupervised or reinforcement learning).

The two most important supervised learning tasks are:

Task	Output Type	Example
Regression	Continuous real-valued output	Predicting house prices
Classification	Discrete label (e.g., +1/-1, class A/B)	Spam detection, image classification

At its core, **supervised learning is curve-fitting** — you have a set of data points and you want to find a function (curve) that fits them well.

2. Essential Mathematical Notation

Before diving deeper, we need a common language.

2.1 Real Numbers ($\mathbb{R}$)

- $\mathbb{R}$ denotes the set of real numbers.
- Examples: 2.3, $\sqrt{\pi}$, -7.6
- **Positive reals** ($\mathbb{R}^+$): real numbers greater than 0.

2.2 D-Dimensional Vectors ($\mathbb{R}^d$)

- $\mathbb{R}^d$ is the set of d -dimensional vectors of real numbers.
- Example in $\mathbb{R}^3$: $(3.6, 5.2, -1.8)$

2.3 Vector Notation

- A vector is denoted by x .
- x_j = the j -th coordinate of vector x .
- $\|x\|$ = the **Euclidean length** (norm) of x .

For a 3D vector $x = (x_1, x_2, x_3)$:

$$\|x\|^2 = x_1^2 + x_2^2 + x_3^2$$

$$\|x\| = \sqrt{x_1^2 + x_2^2 + x_3^2}$$

2.4 Superscripts vs. Subscripts

- **Superscript** $x^{(i)}$ → the i -th vector in a collection.
- **Subscript** x_j → the j -th coordinate of a vector.
- Combined: $x_j^{(i)}$ = the j -th coordinate of the i -th vector.

Example:

$$x^{(1)} = (1, 2, 3), \quad x^{(2)} = (1, 1, 1), \quad x^{(3)} = (7, 7, 8)$$

Then $x_2^{(3)} = 7$ (second element of the third vector).

 **Ambiguity alert:** x_1^2 could mean "square of x_1 " or "the second vector $x^{(1)}$." Context resolves this. When needed, use parentheses: $(x_1)^2$ for the square.

2.5 Indicator Variables

Denoted by $1\{\text{predicate}\}$, it outputs:

- 1 if the predicate is **true**
- 0 if the predicate is **false**

Examples:

$$1\{2 \text{ is even}\} = 1, \quad 1\{2 \text{ is odd}\} = 0$$

This is extremely useful for converting English statements into math.

3. The Supervised Learning Paradigm

3.1 Core Idea: Curve-Fitting

You are given training data:

$$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})$$

- Each $x^{(i)} \in \mathbb{R}^d$ is an **instance** (feature vector).
- Each $y^{(i)}$ is a **label** (target).
 - For regression: $y^{(i)} \in \mathbb{R}$
 - For classification: $y^{(i)} \in \{+1, -1\}$ or a small discrete set.

Goal: Find a model $f : \mathbb{R}^d \rightarrow \mathcal{Y}$ such that:

$$f(x^{(i)}) \approx y^{(i)} \quad \text{for all } i$$

The model f represents the curve. You want the curve to pass as close as possible to all data points.

4. Regression — A Deep Dive

4.1 Problem Definition

Regression = predicting a **continuous real-valued output**.

Example: Predict house price from:

- Number of rooms
- Area
- Distance to metro

Each house is a 3D vector $x \in \mathbb{R}^3$. The label $y \in \mathbb{R}$ is the price.

Training data:

$$(\text{house}_1, \text{price}_1), (\text{house}_2, \text{price}_2), \dots$$

Learning algorithm outputs a model $f : \mathbb{R}^d \rightarrow \mathbb{R}$.

4.2 Loss Function

To evaluate how good a model f is, we use a **loss function**. The most common is the **squared error loss**:

$$L(f) = \frac{1}{n} \sum_{i=1}^n \left(f(x^{(i)}) - y^{(i)} \right)^2$$

Why square?

- Always non-negative (no cancellation between positive and negative errors).
- Penalizes large errors more heavily.
- Differentiable $\rightarrow$ easy to optimize.

Properties:

- $L(f) = 0 \rightarrow$ perfect predictions: $f(x^{(i)}) = y^{(i)}$ for all i .
- Lower loss = better model.

4.3 Linear Models

The most common representation for f is **linear parameterization**:

$$f(x) = w^T x + b$$

Expanded:

$$f(x) = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + b$$

- $w = (w_1, \dots, w_d) \rightarrow$ **weights** (parameters)
- $b \rightarrow$ **bias** (intercept)
- The output is linear in the inputs $\rightarrow$ called a **linear model**.

House price example:

$$\text{price} = w_1 \cdot (\text{rooms}) + w_2 \cdot (\text{area}) + w_3 \cdot (\text{distance}) + b$$

Every choice of (w_1, w_2, w_3, b) gives a different model. The learning algorithm searches for the best parameters.

5. Worked Example: 1D Regression

5.1 Data

i	$x^{(i)}$	$y^{(i)}$
1	1	2.1
2	2	3.9
3	3	6.2
4	6	11.5
5	7	13.9

Here $d = 1$, so $x^{(i)}$ is a scalar (but treated as a 1D vector).

5.2 Two Candidate Models

- Model f : $f(x) = 2x_1$
- Model g : $g(x) = x_1 + 3$

5.3 Predictions

x	y	$f(x) = 2x$	$g(x) = x + 3$
1	2.1	2	4
2	3.9	4	5
3	6.2	6	6
6	11.5	12	9

x	y	$f(x) = 2x$	$g(x) = x + 3$
7	13.9	14	10

5.4 Loss Computation

Loss of f :

$$L(f) = \frac{1}{5} \left[(2 - 2.1)^2 + (4 - 3.9)^2 + (6 - 6.2)^2 + (12 - 11.5)^2 + (14 - 13.9)^2 \right]$$

Loss of g :

$$L(g) = \frac{1}{5} \left[(4 - 2.1)^2 + (5 - 3.9)^2 + (6 - 6.2)^2 + (9 - 11.5)^2 + (10 - 13.9)^2 \right]$$

Clearly, $L(f) \ll L(g)$, so the learning algorithm prefers f .

5.5 Visual Interpretation

- Plotting the data shows a roughly linear trend.
- $f(x) = 2x$ fits the points much better than $g(x) = x + 3$.
- This confirms: **lower loss = better fit**.

6. Worked Example: 3D Regression (House Prices)

6.1 Data (Features: rooms, area, distance)

Rooms	Area	Distance	Price (y)
2	2.0	2.1	5.0
2	2.0	2.1	3.2
4	2.8	0.9	6.6
5	1.8	1.5	9.8
5	3.1	0.8	8.5

Rooms	Area	Distance	Price (y)
4	1.6	1.2	7.2

6.2 Two Candidate Models

- Model f : $f(x) = 2 \cdot \text{rooms} - 0.5 \cdot \text{distance}$
- Model g : $g(x) = \text{rooms} + 2 \cdot \text{distance}$

6.3 Predictions (Approximate)

Actual y	$f(x)$	$g(x)$
5.0	5.0	6.8
3.2	3.0	6.2
6.6	6.6	9.6
9.8	9.5	6.8
8.5	8.5	11.2
7.2	6.9	7.2

6.4 Loss Comparison

Compute:

$$L(f) = \frac{1}{6} \sum (f(x^{(i)}) - y^{(i)})^2$$

$$L(g) = \frac{1}{6} \sum (g(x^{(i)}) - y^{(i)})^2$$

$L(f) < L(g)$, so f is the better model.

6.5 Interpretation

Model f says:

Price increases with number of rooms, decreases with distance to metro.

This is **intuitively reasonable** — a good sign that the model captures real structure.

7. Python Code Implementation

Here's a complete implementation of the concepts covered:

python

```
import numpy as np
import matplotlib.pyplot as plt

# =====
# 1. LOSS FUNCTION: Mean Squared Error (MSE)
# =====
def mse_loss(y_pred, y_true):
    """
    Compute Mean Squared Error between predictions and true labels.

    
$$L(f) = (1/n) * \sum ((f(x_i) - y_i)^2)$$


    Args:
        y_pred: array of predicted values
        y_true: array of true labels
    Returns:
        scalar loss value
    """
    n = len(y_true)
    return np.mean((y_pred - y_true) ** 2)

# =====
# 2. LINEAR MODEL:  $f(x) = w^T x + b$ 
# =====
def linear_model(X, w, b):
    """
    Compute predictions for a linear model.

    Args:
        X: (n, d) feature matrix
        w: (d,) weight vector
        b: scalar bias
    Returns:
        (n,) predictions
    """
```

```

"""
return X @ w + b    # X @ w is matrix-vector product

# =====
# 3. 1D REGRESSION EXAMPLE
# =====

print("=" * 60)
print("1D REGRESSION EXAMPLE")
print("=" * 60)

# Data
X_1d = np.array([[1], [2], [3], [6], [7]], dtype=float) # shape (5, 1)
y_1d = np.array([2.1, 3.9, 6.2, 11.5, 13.9])             # shape (5,)

# Model f:  $f(x) = 2x \rightarrow w = [2], b = 0$ 
w_f, b_f = np.array([2.0]), 0.0
y_pred_f = linear_model(X_1d, w_f, b_f)

# Model g:  $g(x) = x + 3 \rightarrow w = [1], b = 3$ 
w_g, b_g = np.array([1.0]), 3.0
y_pred_g = linear_model(X_1d, w_g, b_g)

# Compute Losses
loss_f = mse_loss(y_pred_f, y_1d)
loss_g = mse_loss(y_pred_g, y_1d)

print(f"Model f predictions: {y_pred_f}")
print(f"Model g predictions: {y_pred_g}")
print(f"Loss of f: {loss_f:.4f}")
print(f"Loss of g: {loss_g:.4f}")
print(f"Better model: {'f' if loss_f < loss_g else 'g'}")

# Visualize
plt.figure(figsize=(8, 5))
plt.scatter(X_1d.flatten(), y_1d, color='black', label='Data', zorder=5)
x_line = np.linspace(0, 8, 100).reshape(-1, 1)
plt.plot(x_line, linear_model(x_line, w_f, b_f), 'b-', label=f'f(x)=2x (loss={loss_f:.2f})')
plt.plot(x_line, linear_model(x_line, w_g, b_g), 'r--', label=f'g(x)=x+3 (loss={loss_g:.2f})')
plt.xlabel('x')
plt.ylabel('y')
plt.title('1D Regression: Comparing Two Linear Models')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('regression_1d.png', dpi=100, bbox_inches='tight')

```

```

plt.show()

# =====
# 4. 3D REGRESSION EXAMPLE (House Prices)
# =====

print("\n" + "=" * 60)
print("3D REGRESSION EXAMPLE (House Prices)")
print("=" * 60)

# Features: [rooms, area, distance]
X_house = np.array([
    [2, 2.0, 2.1],
    [2, 2.0, 2.1],
    [4, 2.8, 0.9],
    [5, 1.8, 1.5],
    [5, 3.1, 0.8],
    [4, 1.6, 1.2],
], dtype=float)
y_house = np.array([5.0, 3.2, 6.6, 9.8, 8.5, 7.2])

# Model f: price = 2*rooms - 0.5*distance + 0*area
# w = [2, 0, -0.5], b = 0
w_fh = np.array([2.0, 0.0, -0.5])
b_fh = 0.0
y_pred_fh = linear_model(X_house, w_fh, b_fh)

# Model g: price = 1*rooms + 0*area + 2*distance
# w = [1, 0, 2], b = 0
w_gh = np.array([1.0, 0.0, 2.0])
b_gh = 0.0
y_pred_gh = linear_model(X_house, w_gh, b_gh)

loss_fh = mse_loss(y_pred_fh, y_house)
loss_gh = mse_loss(y_pred_gh, y_house)

print(f"'Actual':>8} {'f_pred':>8} {'g_pred':>8}")
for a, pf, pg in zip(y_house, y_pred_fh, y_pred_gh):
    print(f"{a:>8.2f} {pf:>8.2f} {pg:>8.2f}")

print(f"\nLoss of f: {loss_fh:.4f}")
print(f"Loss of g: {loss_gh:.4f}")
print(f"Better model: {'f' if loss_fh < loss_gh else 'g'}")

# =====
# 5. FINDING THE OPTIMAL LINEAR MODEL (Closed-Form Solution)

```

```
# =====
print("\n" + "=" * 60)
print("OPTIMAL LINEAR MODEL VIA NORMAL EQUATION")
print("=" * 60)

def fit_linear_regression(X, y):
    """
    Fit a linear model  $f(x) = w^T x + b$  using the normal equation.

    We augment X with a column of ones to absorb the bias b into w.

    Optimal  $w^* = (X^T X)^{-1} X^T y$ 

    Args:
        X: (n, d) feature matrix
        y: (n,) labels
    Returns:
        w: (d+1,) weights (last element is bias)
    """
    n = X.shape[0]
    X_aug = np.hstack([X, np.ones((n, 1))]) # add bias column
    # Normal equation:  $w^* = (X^T X)^{-1} X^T y$ 
    w_opt = np.linalg.inv(X_aug.T @ X_aug) @ X_aug.T @ y
    return w_opt

# Fit optimal model on 1D data
w_opt_1d = fit_linear_regression(X_1d, y_1d)
print(f"Optimal 1D model:  $f(x) = \{w\_opt\_1d[0]:.3f\} * x + \{w\_opt\_1d[1]:.3f\}$ ")

# Compute Loss of optimal model
y_pred_opt_1d = linear_model(X_1d, w_opt_1d[:1], w_opt_1d[1])
loss_opt_1d = mse_loss(y_pred_opt_1d, y_1d)
print(f"Loss of optimal 1D model: {loss_opt_1d:.4f}")

# Fit optimal model on house data
w_opt_house = fit_linear_regression(X_house, y_house)
print(f"\nOptimal house model:")
print(f" price = {w_opt_house[0]:.3f}*rooms + {w_opt_house[1]:.3f}*area "
      f"+ {w_opt_house[2]:.3f}*distance + {w_opt_house[3]:.3f}")
y_pred_opt_house = linear_model(X_house, w_opt_house[:3], w_opt_house[3])
loss_opt_house = mse_loss(y_pred_opt_house, y_house)
print(f"Loss of optimal house model: {loss_opt_house:.4f}")

# =====
# 6. GRADIENT DESCENT (Iterative Optimization)
# =====
```

```

print("\n" + "=" * 60)
print("GRADIENT DESCENT FOR LINEAR REGRESSION")
print("=" * 60)

def gradient_descent(X, y, lr=0.01, epochs=1000):
    """
    Minimize MSE loss using gradient descent.

    Gradients:
        dL/dw = (2/n) * X^T (y_pred - y)
        dL/db = (2/n) * sum(y_pred - y)

    Update:
        w := w - lr * dL/dw
        b := b - lr * dL/db
    """
    n, d = X.shape
    w = np.zeros(d)
    b = 0.0
    losses = []

    for epoch in range(epochs):
        y_pred = X @ w + b
        error = y_pred - y

        # Gradients
        dw = (2 / n) * (X.T @ error)
        db = (2 / n) * np.sum(error)

        # Update
        w -= lr * dw
        b -= lr * db

        # Record Loss
        losses.append(mse_loss(y_pred, y))

    return w, b, losses

# Run gradient descent on 1D data
w_gd, b_gd, losses_gd = gradient_descent(X_1d, y_1d, lr=0.01, epochs=2000)
print(f"Gradient descent result: f(x) = {w_gd[0]:.3f} * x + {b_gd:.3f}")
print(f"Final loss: {losses_gd[-1]:.4f}")
print(f"Normal equation loss: {loss_opt_1d:.4f}")

# Plot Loss curve
plt.figure(figsize=(8, 4))
plt.plot(losses_gd)

```

```
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('Gradient Descent: Loss vs. Epoch')
plt.grid(True, alpha=0.3)
plt.savefig('gd_loss.png', dpi=100, bbox_inches='tight')
plt.show()
```

8. Key Takeaways

1. **Supervised learning = curve-fitting** with labeled data $(x^{(i)}, y^{(i)})$.
2. **Regression** predicts continuous outputs; **classification** predicts discrete labels.
3. **Linear models**: $f(x) = w^T x + b$ — simple, interpretable, foundational.
4. **Loss function (MSE)**: $L(f) = \frac{1}{n} \sum (f(x^{(i)}) - y^{(i)})^2$ measures fit quality.
5. **Learning = optimization**: find parameters (w, b) that minimize loss.
6. **Two solution methods**:
 - **Normal equation** (closed-form): $w^* = (X^T X)^{-1} X^T y$
 - **Gradient descent** (iterative): $w \leftarrow w - \eta \nabla L(w)$
7. **Notation discipline**: superscripts = vector index, subscripts = coordinate index, $1\{\cdot\}$ = indicator.

9. What's Next: Classification

In the next lecture, we move to **classification**, where:

- Labels y are discrete (e.g., $\{+1, -1\}$).
- The model outputs a class decision rather than a real number.
- Loss functions change (e.g., 0-1 loss, cross-entropy).
- Examples: logistic regression, perceptron, SVMs.

The same curve-fitting intuition applies — but now we're fitting a **decision boundary** instead of a regression line.

ask me one liner questions to check if i understand all the topics

Quick-Fire Quiz: Check Your Understanding

Answer these one-liners. I'll correct you after each batch.

Batch 1: Notation

1. What does $\mathbb{R}^d$ represent?
2. In $x_j^{(i)}$, what does the superscript i denote, and what does the subscript j denote?
3. What is $\|x\|^2$ for a 3D vector $x = (x_1, x_2, x_3)$?
4. What does $1\{2 \text{ is odd}\}$ evaluate to?
5. What's the difference between x_1^2 and $(x_1)^2$ in this lecture's notation?

Batch 2: Supervised Learning Basics

6. In one phrase, what is supervised learning at its core?
7. In the tuple $(x^{(i)}, y^{(i)})$, what are $x^{(i)}$ and $y^{(i)}$ called?
8. What is the general goal of a supervised learning algorithm?
9. What are the two most important supervised learning tasks?
10. What type of output does regression produce vs. classification?

Batch 3: Regression & Loss

11. Write the squared error loss formula $L(f)$.
12. Why do we square the error instead of just taking $f(x^{(i)}) - y^{(i)}$?
13. What does $L(f) = 0$ tell you about the model?
14. What is the general form of a linear model?
15. In $f(x) = w^T x + b$, what are w and b collectively called?

Batch 4: Worked Examples

16. In the 1D example, why did model $f(x) = 2x$ beat $g(x) = x + 3$?

17. In the house price example, what did model $f = 2 \cdot \text{rooms} - 0.5 \cdot \text{distance}$ intuitively say about price?
18. Why is a lower loss value preferred?
19. Why can't $L(f)$ ever be negative?
20. If two models have the same predictions on all training points, what can you say about their losses?

Batch 5: Optimization & Code

21. What is the closed-form solution for the optimal weights in linear regression?
22. In gradient descent, what is the update rule for w ?
23. Why do we add a column of ones to X before solving the normal equation?
24. What does the loss curve look like during successful gradient descent?
25. Why might gradient descent be preferred over the normal equation for large datasets?

Go ahead — answer Batch 1 first, and I'll check them before moving on.

you give answers to all the questions

Answers to All Quiz Questions

Batch 1: Notation

1. What does $\mathbb{R}^d$ represent?

The set of all d -dimensional vectors whose coordinates are real numbers.

2. In $x_j^{(i)}$, what does the superscript i denote, and what does the subscript j denote?

Superscript i = the i -th vector in a collection; subscript j = the j -th coordinate of that vector.

3. What is $\|x\|^2$ for a 3D vector $x = (x_1, x_2, x_3)$?

$$\|x\|^2 = x_1^2 + x_2^2 + x_3^2$$

4. What does $1\{2 \text{ is odd}\}$ evaluate to?

0 (since "2 is odd" is false).

5. What's the difference between x_1^2 and $(x_1)^2$ in this lecture's notation?

x_1^2 is ambiguous (could be the square of coordinate x_1 , or the second vector $x^{(1)}$); $(x_1)^2$ explicitly means the square of the first coordinate. Context usually resolves the ambiguity.

Batch 2: Supervised Learning Basics

6. In one phrase, what is supervised learning at its core?

Curve-fitting.

7. In the tuple $(x^{(i)}, y^{(i)})$, what are $x^{(i)}$ and $y^{(i)}$ called?

$x^{(i)}$ = instance (feature vector); $y^{(i)}$ = label (target).

8. What is the general goal of a supervised learning algorithm?

Find a model f such that $f(x^{(i)}) \approx y^{(i)}$ for all training points (i.e., minimize loss).

9. What are the two most important supervised learning tasks?

Regression and classification.

10. What type of output does regression produce vs. classification?

Regression $\rightarrow$ continuous real-valued output ($\mathbb{R}$); classification $\rightarrow$ discrete label (e.g., $\{+1, -1\}$).

Batch 3: Regression & Loss

11. Write the squared error loss formula $L(f)$.

$$L(f) = \frac{1}{n} \sum_{i=1}^n \left(f(x^{(i)}) - y^{(i)} \right)^2$$

12. Why do we square the error instead of just taking $f(x^{(i)}) - y^{(i)}$?

Squaring makes all terms non-negative (prevents positive/negative errors from cancelling) and penalizes large errors more heavily.

13. What does $L(f) = 0$ tell you about the model?

The model predicts perfectly: $f(x^{(i)}) = y^{(i)}$ for all i .

14. What is the general form of a linear model?

$$f(x) = w^T x + b$$

15. In $f(x) = w^T x + b$, what are w and b collectively called?

Parameters (weights w and bias b).

Batch 4: Worked Examples

16. In the 1D example, why did model $f(x) = 2x$ beat $g(x) = x + 3$?

Because f had a much smaller MSE loss — its predictions were closer to the true labels than g 's.

17. In the house price example, what did model $f = 2 \cdot \text{rooms} - 0.5 \cdot \text{distance}$ intuitively say about price?

Price increases with number of rooms and decreases with distance to the metro — intuitively reasonable.

18. Why is a lower loss value preferred?

Lower loss means predictions are closer to the true labels, i.e., a better fit to the data.

19. Why can't $L(f)$ ever be negative?

Because each term is a square, $(f(x^{(i)}) - y^{(i)})^2 \geq 0$, and the average of non-negative terms is non-negative.

20. If two models have the same predictions on all training points, what can you say about their losses?

Their losses are identical (loss depends only on predictions vs. true labels).

Batch 5: Optimization & Code

21. What is the closed-form solution for the optimal weights in linear regression?

$$w^* = (X^T X)^{-1} X^T y$$

(after augmenting X with a column of ones to absorb the bias).

22. In gradient descent, what is the update rule for w ?

$$w := w - \eta \cdot \frac{\partial L}{\partial w}$$

For MSE: $w := w - \eta \cdot \frac{2}{n} X^T (y_{\text{pred}} - y)$

23. Why do we add a column of ones to X before solving the normal equation?

To absorb the bias b into the weight vector w , so we can solve for all parameters in one matrix equation.

24. What does the loss curve look like during successful gradient descent?

It decreases monotonically (or nearly so) over epochs, ideally converging to a minimum.

25. Why might gradient descent be preferred over the normal equation for large datasets?

Computing $(X^T X)^{-1}$ costs $O(d^3)$ and is infeasible for very large d ; gradient descent scales better and works for models where no closed-form solution exists.

Score Yourself

Score	Verdict
23–25	Excellent — you've mastered the lecture
18–22	Good — review the missed topics
12–17	Fair — re-read the notes on weak areas
< 12	Needs work — go through the full notes again

Want me to generate more questions on any specific topic (e.g., gradient descent math, normal equation derivation, or classification preview)?